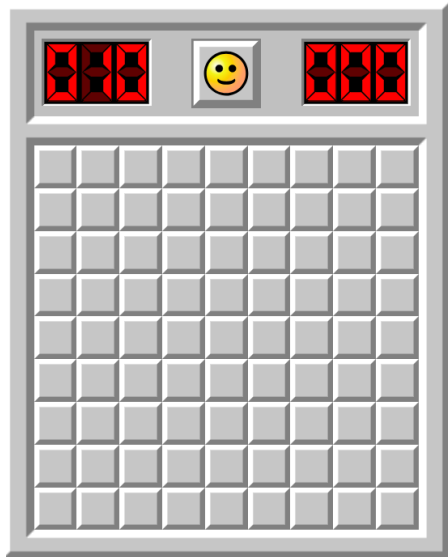


Minesweeper Game Project

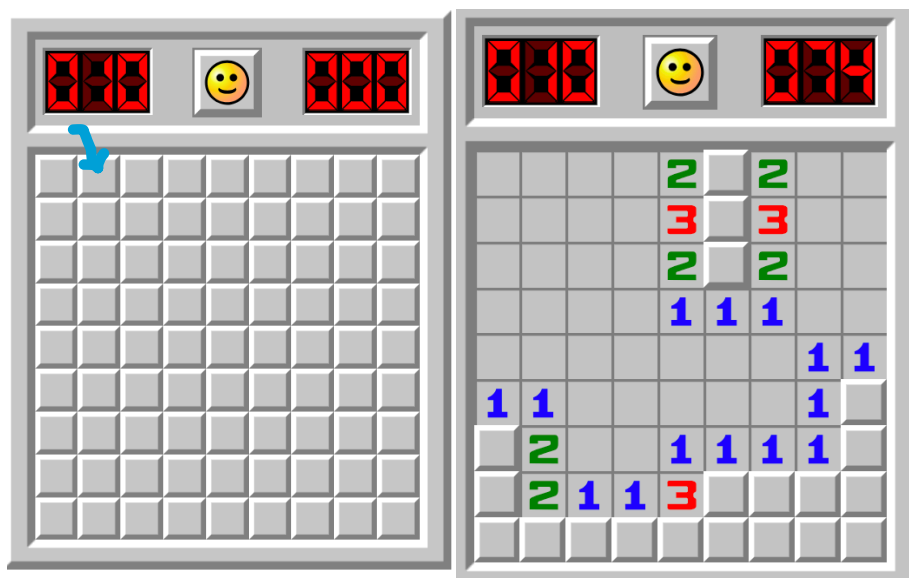
Introduction to minesweeper game

In minesweeper game, the board is divided into cells, with mines randomly distributed. To win, you need to open all the cells. The number on a cell shows the number of mines adjacent to it. Using this information, you can determine cells that are safe, and cells that contain mines. Cells suspected of being mines can be marked with a flag using the right mouse button. See <https://minesweeper.online/game/2514994168>.

- (1) Start with an empty board, where the number in the top left is the number of mines, while the top right is the number of seconds used.



- (2) Pick any cell that is unclicked. For example, click the first row and second cell, we get the layout in the right-hand side.



From the above layout, we can find out mines 1, 2, 3 since these are only available slots to get three mines (see numbers 3 to the left/right of these cells). To make the connection, I highlight those cells. Similarly,

- Mine 4 is found through its upper left neighbor with value 1.
- Mine 5 is found through its upper right neighbor with value 1.
- After Mine 5 is found, Mine 6 is found through its upper right neighbor with value 2.
- Mine 7 is found through its upper left neighbor with value 1.



- The cell under the cell with mine 4 cannot have a mine; otherwise, the cell to the left of 4 or to the bottom left of 4 must have a value larger than 1.



This cell cannot contain a mine; otherwise, its neighbors in circle must have values larger than 1. Hence it is safe to open (click) this cell.



This cell cannot contain a mine; otherwise, its neighbors in circle must have values larger than 1. Hence it is safe to open this cell.

After open the above two cells, we get the following layout.



- (4) The cells in circles are safe to open, otherwise, the highlighted cells with known mines must have higher values.



- (5) After opening the above circled cells, we have the following layout.



Then since the highlighted cell at the next-to-last row and the third column has value 1 and all its other neighbors are cleared (no mines), so its bottom right neighbor has a mine, call it 8.

Furthermore, the right neighbor of the cell with mine 8 is safe to open, otherwise, its top upper neighbor should have value 2.



This cell cannot contain a mine; otherwise, its neighbors in circle must have values larger than 1. Hence it is safe to open (click) this cell.

(6) After opening the above cell, we find out mine 9.



Adaption of minesweeper game to one-dimensional grid

Our program is a simplification of minesweeper game.

- (1) Use textual output, no GUI (Graphic User Interface) design.
- (2) Use only one row, not grids.
- (3) When you choose a cell, ensure it has a correct index and is not chosen before, ie, not yet checked and no number is displayed in that cell.
 - (3a) if it has a mine, print "Mine explodes! Game ends." The game finishes.
 - (3b) otherwise, display a number showing the number of mines in its neighbors. If all the cells without mines are discovered, print "Congratulations! Find out all the cells without mines." The game ends.

Here is a sample run of our program. Suppose there are 7 cells with 2 randomly placed mines.

- (1) In the very beginning, show 7 cells, the first one starts from zero.

0	1	2	3	4	5	6

- (2) Ensure a valid cell is chosen.

```
Round 1: Please choose a cell (the first starts from 0): -1
Enter an integer in [0, 6]: -2
Enter an integer in [0, 6]: 7
Enter an integer in [0, 6]: 0
Mine explodes! Game ends.
```

- (3) Display the answer when the game ends.

The answer is

0	1	2	3	4	5	6

Here is another run.

- (1) In the very beginning, display an empty layout.

0	1	2	3	4	5	6
+	+	+	+	+	+	+
+	+	+	+	+	+	+

- (2) Choose a valid-indexed and unopened cell to open. We enter 0.

Round 1: Please choose a cell (the first starts from 0): 0

0	1	2	3	4	5	6
+	+	+	+	+	+	+
0						
+	+	+	+	+	+	+

- (3) Choose a valid-indexed and unopened cell to open. Enter 1 since from the number of neighbors displayed in Cell 0, we learn that Cell 0 has no mine in its neighbor; that is, Cell 1 has no mine.

Round 2: Please choose a cell (the first starts from 0): 1

0	1	2	3	4	5	6
+	+	+	+	+	+	+
0	0					
+	+	+	+	+	+	+

- (4) Choose a valid-indexed and unopened cell to open. Note that as long as (asa, while) you enter any invalid index or opened-already cell, the user is asked to re-enter.

Round 3: Please choose a cell (the first starts from 0): 0

Cell 0 is already chosen. Please re-enter: -1

Enter an integer in [0, 6]: 1

Cell 1 is already chosen. Please re-enter: 2

0	1	2	3	4	5	6
+	+	+	+	+	+	+
0	0	0				
+	+	+	+	+	+	+

- (5) Enter 3 since by the value of Cell 2, we learn that Cell 2 has no neighbor has a mine. Thus, Cell 3 is free of a mine.

Round 4: Please choose a cell (the first starts from 0): 3

0	1	2	3	4	5	6
0	0	0	1			

(6) Enter 5 since by the value of Cell 3, we learn that Cell 3 has a mine in its neighbors.
Since the left neighbor 3 has no mine, the mine must be in its right neighbor 4.

Round 5: Please choose a cell (the first starts from 0): 5

0	1	2	3	4	5	6
0	0	0	1		2	

(7) Now, every cell without a mine is chosen. The game ends successfully. Print the following prompt.

Congratulations! Find out all the cells without mines.

(8) After a game ends, print out the correct answer.

The answer is

0	1	2	3	4	5	6
0	0	0	1	*	2	*

Here is another run.

0	1	2	3	4	5	6

Round 1: Please choose a cell (the first starts from 0): 0

0	1	2	3	4	5	6
1						

Round 2: Please choose a cell (the first starts from 0): 2

0	1	2	3	4	5	6
1		1				

Round 3: Please choose a cell (the first starts from 0): 3

0	1	2	3	4	5	6
1		1	1			

Round 4: Please choose a cell (the first starts from 0): 5

0	1	2	3	4	5	6
1		1	1		1	

Round 5: Please choose a cell (the first starts from 0): 6

0	1	2	3	4	5	6
1		1	1		1	0

Congratulations! Find out all the cells without mines.

The answer is

0	1	2	3	4	5	6
1	*	1	1	*	1	0

Field class

header file of Field class, saved in Field.hpp

```
//#pragma once //not work in older version of C++
#ifndef FIELD_H //include guard
#define FIELD_H
#include<string>
class Field {
    private:
        int* cells;
        bool* checked;
        int size;
        int num_mines;

    public:
        Field(); //constructor
        Field(int size, int num_mines);
        ~Field(); //destructor
        std::string to_string() const;
        std::string answer_string() const;
        std::string empty_layout_string() const;
        int get_size() const;
        int get_num_mines() const;
        int neighbor_mines(int index) const;
        void mark_checked(int index);
        bool is_checked(int index) const;
        bool has_mine(int index) const;
};
#endif
```

Define Field class, saved in Field.cpp

data members of Field class

- (1) an array of integers called **cells**: if a cell has a mine, its value is 1, otherwise, it is zero.
- (2) an array of bool called **checked**: if a cell is checked already, its value is true, otherwise, it is false. In the beginning, every element of array checked is false. This array tells us which cells have been checked, thus to avoid to check a cell more than once.
- (3) variable **size** is the capacity of arrays **cells** and **checked**.
- (4) variable **num_mines** is the number of mines in this field.

constructors of Field class

- (1) Default constructor Field():

- Initialize **size** to be 7 and **num_mines** to be 2.
- Set **cells** to be an array with capacity **size**, elements are either 0 or 1, where number of 1's equals **num_mines**.
- Set **checked** to be an array with capacity **size**, each element is set to be false.

(2) Non-default constructor `Field(int size, int num_mines)`:

- If given parameter **size** is ≤ 3 , set data member **size** to be 7, otherwise, use given parameter **size** to initialize data member **size**.
- If given parameter **num_mines** < 2 , set data member **num_mines** to be 2, otherwise, use given parameter **num_mines** to initialize data member **num_mines**.
- If **num_mines** $\geq$ **size**, which means mines are more than the number of cells, then set **num_mines** to be one third ($1/3$) of **size**.
- Set **cells** to be an array with capacity **size**, elements are either 0 or 1, where number of 1's equals **num_mines**.
- Set **checked** to be an array with capacity **size**, each element is set to be false.

(3) Destructor `~Field()`: called after an object of `Field` is no longer needed.

- Release dynamic memory allocated to data member **cells**. Avoid dangling pointer problem by set cells to `nullptr`.
- Do similar things for data member **checked**.

(4) Optional: After you finish defining constructors, you may find out that they have redundant codes. To reduce code redundancy, you can define non-default constructor only. Then let default constructor call a non-default constructor using constructor delegation, which needs C++11 or higher.

Methods

(5) `to_string()`: returns a string representing a tabular format of cells with labels. If a cell does not have a mine and is checked, then display the number of its neighbors, otherwise, that is, a cell does not have a mine but is not checked or a cell has a mine, display an empty cell. This method is provided as follows; put the code in `Field.cpp`.

```
//Since get_labels is not a must-provided
//functionality for Field class,
//it is not a method in Field class.
//Hence it does not have Field:: in front of its name.
//Function get_labels is called by
//to_string method of Field class.
string get_labels(int size) {
    string str;
    for (int i = 0; i < size; i++) {
        str += " ";
        if (i < 10)
```

```

        str += " " + std::to_string(i);
    else str += to_string(i);
    str += " ";
}
str += "\n";
return str;
}

//Since get_separate_line is not a must-provided
//functionality for Field class,
//it is not a method in Field class.
//Hence it does not have Field:: in front of its name.
//Function get_separate_line is called by
//to_string method of Field class.
string get_separate_line(int size) {
    string str = "+";
    for (int i = 0; i < size; i++)
        str += "-----+";
    str += "\n";
    return str;
}

string Field::to_string() const {
    string str = get_labels(size);
    str += get_separate_line(size);

    str += "|";
    for (int i = 0; i < size; i++) {
        if (cells[i] == 0)
        {
            if (checked[i] == true)
                str += " " + std::to_string(neighbor_mines(i)) +
                    " |";
            else str += "      |";
        }
        else //must have a mine
            str += "      |";

        //The above if-else can be simplified.
        //Idea: if a cell has no mine and is checked,
        //display its neighboring information,
        //otherwise, that is,
        //a cell has a mine or
        //it does not have a mine but is not checked yet,
        //do not display anything in that cell.
        //The code is as follows.
        //if (cells[i] == 0 &&

```

```

        //      checked[i] == true)
        //      str += "    " + std::to_string(neighbor_mines(i)) +
        //              "    |";
        //else str += "    |";
    }

    str += "\n";

    str += get_separate_line(size);

    return str;
}

```

- (6) `answer_string()`: returns a string representing a tabular format of cells with labels. If a cell does not have a mine, then display the number of its neighbors, otherwise, that is, display an asterisk, which represents a mine.
- (7) `empty_layout_string()`: return a string representing a tabular format of cells with labels. Each cell is empty. This is called in the beginning of a game.
- (8) `get_size()`: return the value of data member **size** in current Field object.
- (9) `get_num_mines()`: return the value of data member **num_mines** in current Field object.
- (10) `neighbor_mines(int index)`: if current cell indexed at index does not contain a mine, return number of mines in its left (if any) and right neighbor (if any), otherwise, return -1.
- (11) `mark_checked(int index)`: if index is valid, that is, index is larger than or equal to 0 and is smaller than data member **size**, set element at the corresponding index of **checked** array to be true.
- (12) `is_checked(int index) const`: if the element at the corresponding index of **checked** array is true, return true, otherwise, return false. That is, return the value of element at the corresponding index of **checked** array.
- (13) `has_mine(int index) const`: if the element at the corresponding index of cells array contains a mine, that is, `cells[index]` is 1, return true, otherwise, return false.
- (14) Some methods have `const` in their headers since they do not change the value of current Field object.

Test Field class

- (1) Create a project called minesweeper.
- (2) Create file Field.hpp, no modification is needed.
- (3) Create file Field.cpp. Its method to_string is provided. Note that functions get_labels and get_separate_line are not methods of class Field. These two functions are not operations on data members of Field class and are not must-provide-feature for the class. Functions get_labels and get_separate_line can be used by methods to_string, empty, and answer_string to reduce code redundancy.

You may think a class as a company, and a method of that class is like an employee of that company. A function used by a method of a class is like a contractor hired by that employee to help, but that contractor (function) does to belong to the company (class).

header file of Game, saved in Game.hpp

```
//#pragma once
#ifndef GAME_H
#define GAME_H
#include "Field.hpp"
class Game {
    private:
        Field area;
    public:
        Game();
        Game(int size, int num_mines);
        int input() const;
        void play();
};
#endif
```

Data member of Game.hpp

A game is played in a field object, call it area.

Implementation of constructors and methods declared in Game.hpp, saved in Game.cpp

Constructors

- (1) Default constructor Game(): instantiate area by default constructor of Field.
- (2) Non-default constructor Game(int size, int num_mines): instantiate area by non-default constructor of Field(int size, int num_mines).
- (3) No destructor for Game class since we do not apply for dynamic allocated memory in this class.
- (4) input(): **ensure** to enter a valid index num whose cell is not checked yet, return num.
Hint: which method of area to call to find out whether a cell in it is checked or not?

Here are the steps.

First, print "Please choose a cell (the first starts from 0): " and input an int for num from console.

Keep asking for input num as long as num is not in [0, size) or the corresponding cell is checked, where size is the capacity of data member **area**.

begin

If num is out of range, that is, not in [0, size), prompt "Enter an integer in [0, size-1]: ", replace size-1 by the actual value of given parameter size in this method.

Otherwise, the cell must be checked (why??), prompt "Cell ... is already chosen. Please re-enter: ", where ... is the value of num

Re-enter num

end

return num

(5) method play(): code to show how to play a game. Starting with an empty layout of field, keep choosing a valid index for an unchecked cell until one of the following happens.

(a) A cell with a mine is chosen, in that case, print "Mine explodes! Game ends."

(b) All cells without mines are found out, in that case, print "Congratulations! Find out all the cells without mines."

(c) After the game, print the layout of fields where mines are marked as * and non-mines are marked by its number of neighbor(s).

Code to Test Game, which in turn calls Field

```
#include <iostream>
#include "Field.hpp"
#include "Game.hpp"
using namespace std;

int main()
{
    //call default constructor using
    Game minesweeper;
    //Game minesweeper(9, 3);
    minesweeper.play();
    return 0;
}
```


Contents of makefile

```
# This is an example Makefile for dig mine project. This
# program uses Field, Game, and TestGame modules.
# Typing 'make' or 'make run' will create the executable file.
#

# define some Makefile variables for the compiler and compiler
# flags
# to use Makefile variables later in the Makefile: $()
#
# -g      adds debugging information to the executable file
# -Wall   turns on most, but not all, compiler warnings
#
# for C++ define  CC = g++
CC = g++ -std=c++11
#CFLAGS   = -g -Wall

# typing 'make' will invoke the first target entry in the file
# (in this case the default target entry)
# you can name this target entry anything, but "default" or
# "all"
# are the most commonly used names by convention
#
all: run

# To create the executable file run we need the object files
# TestGame.o, Field.o
run: Field.o TestGame.o Game.o
    $(CC) -o run TestGame.o Field.o Game.o

# To create the object file TestGame.o, we need the source
# files TestGame.cpp and Field.hpp
#
TestGame.o: TestGame.cpp
    $(CC) -c TestGame.cpp

Player.o: Player.cpp
    $(CC) -c Player.cpp

# To create the object file Field.o, we need the source
# files Field.cpp, Field.hpp
Field.o: Field.cpp
    $(CC) -c Field.cpp

# To start over from scratch, type 'make clean'. This
# removes the executable file, as well as old .o object
```

```
# files and *~ backup files:
#
clean:
    $(RM) run *.o *~
```

How to run the project

Call **make** (with return key) command in command line, then type in **run** (with return key), the executable file specified in makefile.

Task A (20 points)

Implement constructors of Field class.

Task B (30 points)

Write code for not-implemented method of Field class.

Task C (20 points)

Implement constructors and input method of Game class.

Task D (30 points)

Implement play method of Game class.

Common questions of the project

Q: What is difference between #pragma once and include guard #ifndef ... #define ... #endif?

A: Both #pragma once and #ifndef ... #define ... #endif can make sure a header file will not be included more than once. For example, suppose Hare and Tortoise are inherited from Animal class, then Animal.hpp are included in both Hare.hpp and Tortoise.hpp. Then when we have a competition involves Hare and Tortoise, Animal.hpp would have been included more than once, as a result, data members and function members in Animal.hpp are declared more than once, which causes compilation problem.

The difference is, unlike include guard, #pragma once may not work with every version of C++.

Q: In the worst case that the files are not executed, can we put everything in one file and run it?

A: No, that is not a good idea. A project can involve a lot of tasks, different teams will work on different tasks. Also, if you put everything in one file, it is very hard to debug or to expand.

Furthermore, it does not fit for the requirement of object oriented programming (oop), which uses class and objects. One important oop feature is encapsulation, which combines data and operations on the data. It is clear that classes Field and Game are different classes, with different data members and different operations.

Q: What are we using makefile for?

A: makefile is a way to compile and link multiple files in a project to generate runnable target. It uses scripts saved in a file called makefile. If some files are not modified, then we do not need to recompile them again, this will save time.

Every time some files in the project is modified, run make command. Then run the runnable target. If you want to clean the generated object files and runnable target, run command make clean